



Setup, Integrations & Deployment Guide

Running it with uv, every setting and what to change for production, putting it on GitHub, connecting Gmail, Calendar, Notion, Groq and n8n (WhatsApp, Telegram, Slack), and deploying it — step by step.

Version 0.2

Stack LangGraph · MCP · CrewAI · FastAPI · React · Obsidian

Platforms macOS (primary) · Linux · Docker

Generated 26 September 2026

Contents

1–4 get it running. 5 puts it on GitHub. 6–9 connect your apps. 10–14 deploy and operate it.

- 1 What you need — tools, hardware, the one decision that matters
- 2 Run it with uv — first run, then daily use
- 3 Configuration reference — every variable, dev vs production, what to replace
- 4 Verify it — doctor, tests, evals, browser checks
- 5 Put it on GitHub — first push, CI, keeping secrets out
- 6 Fast model (Groq) — the biggest speed upgrade, free
- 7 Gmail & Google Calendar
- 8 Notion
- 9 n8n: WhatsApp, Telegram, Slack, Sheets — concepts, securing, five workflow recipes, inbound auto-reply
- 10 Deployment — always-on Mac, Docker, VPS, tunnels
- 11 Pre-launch checklist
- 12 Operating it — backups, upgrades, rollback, what to watch
- 13 Troubleshooting
- 14 Changing it yourself

THE ONE THING TO KNOW FIRST

By default Synapse runs entirely on your machine with a local model (Ollama). That is private and free, but a local 7B model takes roughly 30–70 seconds to plan a task. A free Groq key (section 6) brings that to 2–5 seconds *and* plans better, with the local model kept as an automatic fallback. If you do one optional thing, do that one.

Conventions in this guide

- Commands are for the macOS Terminal (zsh). Paste **one line at a time**; lines starting with `#` are comments.
- `~/Desktop/synapse` is the project folder. Replace it if yours is elsewhere.
- Anything in `<angle brackets>` is a placeholder you must replace, brackets included.

1 What you need

Install these once.

TOOL	WHY	INSTALL (MACOS)
Homebrew	Installs everything below	<code>/bin/bash -c "\$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"</code>
uv	Python, environments, packages	<code>brew install uv</code>
Node 20+	Builds the dashboard	<code>brew install node</code>
Ollama	Local model server	<code>brew install ollama</code>
git	Version control, GitHub	<code>xcode-select --install</code> (or <code>brew install git</code>)
GitHub CLI (optional)	One-command repo creation	<code>brew install gh</code>
Obsidian (optional)	To browse the vault	<code>brew install --cask obsidian</code>

You do **not** need to install Python yourself: uv downloads the right version (3.12) for the project.

Hardware

16 GB RAM (your MacBook Air M4) runs a 7B model comfortably; 8 GB works but swaps. Apple Silicon uses the GPU automatically. Disk: about 6 GB for models, plus your vault.

NEVER RUN OLLAMA ON A CLOUD VM

Cloud CPUs manage a few tokens per second, which makes the assistant unusable. Anywhere other than your own Mac, use a hosted model (sections 3 and 6).

2 Run it with uv

First run takes about 10 minutes, mostly downloads. After that it is two commands.

2.1 Start the model server

Terminal 1 — leave it running (or run `brew services start ollama` once and it starts at login):

```
ollama serve
```

Terminal 2 — download the models:

```
ollama pull qwen2.5:7b-instruct # planner/writer, ~4.7 GB
ollama pull nomic-embed-text   # memory search, ~270 MB, optional
```

2.2 Install the project

```
cd ~/Desktop/synapse
uv sync --extra dev --extra google
```

`uv sync` downloads Python 3.12 if needed, creates `.venv/`, and installs exactly the versions pinned in `uv.lock`. The extras: `dev` = tests, `google` = Gmail and Calendar, `crew` = the CrewAI research crew (large; add `--extra crew` if you want it).

2.3 Create your settings file

```
cp .env.example .env
open -e .env
```

At minimum set `SYNAPSE_VAULT_PATH` to the absolute path of your Obsidian vault (e.g. `/Users/<you>/Documents/MyVault`). Section 3 explains every other line.

2.4 Build the dashboard

```
cd web
npm install
npm run build
cd ..
```

Repeat only when you change something under `web/src`. For UI work, `npm run dev` in `web/` serves on port 5173 with hot reload.

2.5 Check and start

```
uv run synapse doctor
uv run synapse-api
```

Open **http://127.0.0.1:8000**. **doctor** checks Ollama, the model, every MCP server, Google, and prints the vault path it will really use — compare it with Obsidian.

DAILY USE

`cd ~/Desktop/synapse` then `uv run synapse-api` (with Ollama running). That is all. `uv run` always uses the project's own environment — no `source .venv/bin/activate` needed. Section 10.1 makes even this automatic.

2.6 Useful uv commands

COMMAND	WHAT IT DOES
<code>uv run synapse run "<goal>"</code>	Run one goal in the terminal; approvals are asked there
<code>uv run pytest -q</code>	Test suite
<code>uv run python -m evals.run</code>	Behaviour evaluations
<code>uv sync --extra dev --extra google</code>	Re-install after pulling changes
<code>uv add <package></code>	Add a dependency (updates pyproject and uv.lock)
<code>uv lock --upgrade</code>	Upgrade all pinned versions (then re-test)
<code>rm -rf .venv && uv sync ...</code>	Rebuild a broken environment from scratch

3 Configuration reference

Every variable in `.env`: what it does, a good development value, and what to use in production.

3.1 What you must replace

PLACEHOLDER IN THE EXAMPLE	REPLACE WITH
<code>SYNAPSE_VAULT_PATH=.</code> / <code>SynapseVault</code>	The absolute path of the folder Obsidian has open. Wrong path = notes you never see.
<code># SYNAPSE_API_TOKEN=</code>	Uncomment and set to the output of <code>openssl rand -hex 32</code> before anything but your Mac can reach the port.
<code>SYNAPSE_FAST_API_KEY=</code>	Your Groq key (section 6).
<code>ntn...</code>	Your Notion integration secret (section 8).
<code>https://your-n8n/webhook/abc</code>	The <i>production</i> URL of each n8n Webhook node (section 9). Remove examples you do not use.
<code># SYNAPSE_N8N_SECRET=</code>	<code>openssl rand -hex 24</code> , and the same value in n8n's Header Auth credential.

3.2 Model

VARIABLE	DEFAULT	MEANING · PRODUCTION ADVICE
<code>SYNAPSE_LLM_PROVIDER</code>	ollama	Main model backend. On your Mac keep <code>ollama</code> and add the fast provider. On a VPS (no Ollama) set <code>openai</code> and fill the two lines below.
<code>SYNAPSE_MODEL</code>	qwen2.5:7b-instruct	Main model name. With <code>openai</code> provider on Groq: <code>llama-3.3-70b-versatile</code> .
<code>SYNAPSE_OPENAI_BASE_URL</code> <code>SYNAPSE_OPENAI_API_KEY</code>	Groq URL / unset	Only read when the provider is <code>openai</code> . Any OpenAI-compatible endpoint works (Groq, Cerebras, OpenRouter, OpenAI).
<code>SYNAPSE_EMBED_MODEL</code>	nomic-embed-text	Ollama embedding model for memory search. Without Ollama, memory falls back to keyword search — fine.
<code>SYNAPSE_OLLAMA_URL</code>	localhost:11434	Docker overrides it to <code>host.docker.internal</code> automatically.
<code>SYNAPSE_LLM_TIMEOUT</code>	180	Seconds per model call. Local: 180. Hosted: 60, so a stuck call fails fast and falls back.
<code>SYNAPSE_FAST_BASE_URL</code> <code>SYNAPSE_FAST_MODEL</code> <code>SYNAPSE_FAST_API_KEY</code>	unset	Recommended everywhere. Used for planning, writing and answering; local Ollama is the automatic fallback when the quota runs out.
<code>SYNAPSE_NUM_CTX</code>	8192	Ollama context window. Raise to 16384 only if long notes get cut; it slows every call.
<code>SYNAPSE_KEEP_ALIVE</code>	30m	How long Ollama keeps the model loaded. <code>24h</code> on an always-on Mac avoids reload pauses.
<code>SYNAPSE_PLAN_MAX_TOKENS</code>	700	Plan length cap. Leave.
<code>SYNAPSE_GEN_MAX_TOKENS</code>	1200	Longest answer or note. Raise to 2000 for long reports.

3.3 Paths

VARIABLE	DEFAULT	MEANING · PRODUCTION ADVICE
SYNAPSE_VAULT_PATH	./SynapseVault	Absolute path, always. In Docker it is fixed to <code>/vault</code> ; set the host folder with <code>VAULT_PATH=</code> instead.
SYNAPSE_DATA_DIR	./data	All state: runs, traces, memory, schedules, checkpoints and the Google token. Absolute path in production, and back it up (section 12).

3.4 Budgets and guard rails

VARIABLE	DEFAULT	MEANING · PRODUCTION ADVICE
SYNAPSE_TOOL_TIMEOUT	30	Seconds before a tool call is abandoned (read-only tools are retried once). 45 if research often times out.
SYNAPSE_MAX_STEPS	6	Longest plan. 6–8 is right; longer plans are slower and more fragile, not smarter.
SYNAPSE_MAX_REPLANS	1	Re-plans after a failed step. Keep 1: a second replan rarely succeeds where the first failed.
SYNAPSE_MAX_LLM_CALLS	20	Hard per-run budget; the loop guard. With a paid provider this is also your cost ceiling per request.
SYNAPSE_PARALLEL_STEPS	2	Independent read-only steps run together. 3 on a hosted model; keep 2 on local Ollama.
SYNAPSE_CREW_TIMEOUT	420	Seconds for a CrewAI research report. 600 local, 300 hosted.

3.5 Features

VARIABLE	DEFAULT	MEANING · PRODUCTION ADVICE
SYNAPSE_ENABLE_CREW	true	Needs the <code>crew</code> extra. Set <code>false</code> on a small VPS (large install, slow).
SYNAPSE_CREW_CRITIC	false	Adds a critic agent to the crew; roughly doubles its time.
SYNAPSE_ENABLE_WEB	true	Web search and page fetch. Keep on.
SYNAPSE_ENABLE_SCHEDULER	true	Runs scheduled goals — only while the API is running.
SYNAPSE_SCHEDULER_INTERVAL	30	Seconds between schedule checks. Leave.

3.6 API and security

VARIABLE	DEFAULT	MEANING · PRODUCTION ADVICE
SYNAPSE_HOST	127.0.0.1	Keep 127.0.0.1. Reach it from outside through a tunnel or reverse proxy, never by binding 0.0.0.0 on a public machine. (Docker sets 0.0.0.0 inside the container but publishes only to 127.0.0.1.)
SYNAPSE_PORT	8000	Change only if 8000 is taken.
SYNAPSE_API_TOKEN	unset	Required before exposure. Without it every route is open — including the ones that send email. The dashboard asks for it once and keeps it in the browser.

3.7 Integrations

VARIABLE	DEFAULT	MEANING · PRODUCTION ADVICE
----------	---------	-----------------------------

SYNAPSE_NOTION_TOKEN	unset	Enables the Notion tools (section 8).
SYNAPSE_N8N_WEBHOOKS	unset	<code>name=url</code> pairs, comma-separated, no spaces. An <i>allowlist</i> : Synapse can call only these. Use production URLs (<code>/webhook/</code> , not <code>/webhook-test/</code>) over https.
SYNAPSE_N8N_SECRET	unset	Sent as header <code>X-Synapse-Secret</code> on every call so n8n can reject anyone else who finds the URL.

SECRETS

`.env` and `data/` (which holds `google_token.json`) are git-ignored — keep it that way. On a server, set them as environment variables or a root-only `.env` (`chmod 600 .env`). If a key ever lands in a commit, **rotate it** at the provider; deleting the file is not enough.

3.8 A production `.env` for your Mac

```
SYNAPSE_LLM_PROVIDER=ollama
SYNAPSE_MODEL=qwen2.5:7b-instruct
SYNAPSE_KEEP_ALIVE=24h
SYNAPSE_FAST_BASE_URL=https://api.groq.com/openai/v1
SYNAPSE_FAST_MODEL=llama-3.3-70b-versatile
SYNAPSE_FAST_API_KEY=<gsk_...>
SYNAPSE_VAULT_PATH=/Users/<you>/Documents/<YourVault>
SYNAPSE_DATA_DIR=/Users/<you>/Desktop/synapse/data
SYNAPSE_MAX_STEPS=8
SYNAPSE_API_TOKEN=<output of openssl rand -hex 32>
SYNAPSE_N8N_WEBHOOKS=telegram=https://<n8n-host>/webhook/<id1>,whatsapp=https://<n8n-host>/webhook/<id2>
SYNAPSE_N8N_SECRET=<output of openssl rand -hex 24>
```

On a VPS the only differences: `SYNAPSE_LLM_PROVIDER=openai` , `SYNAPSE_MODEL=llama-3.3-70b-versatile` , `SYNAPSE_OPENAI_API_KEY=<gsk_...>` , `SYNAPSE_ENABLE_CREW=false` , and no Ollama lines.

4 Verify it

Run these after install and after every change you make.

```
uv run synapse doctor          # Ollama, model, MCP servers, Google, vault path
uv run pytest -q               # ~50 tests: runtime, tools, API – real graph and MCP, scripted
                               # model
uv run python -m evals.run     # 16 behaviour scenarios; report shown in the Evals tab
uv run ruff check synapse evals tests
```

BROWSER CHECKS (OPTIONAL)

Loads the built dashboard in real Chromium and checks console errors, labels, keyboard use, layout at phone width and responsiveness. Start the API first, then in another terminal:

```
uv pip install playwright
uv run playwright install chromium
uv run python scripts/browser_check.py
```

FIRST REAL TEST

1. Type *Hi* — it should answer in a few seconds with no plan.
2. Type *Build a study roadmap for AI engineering and save it to my vault* — open “Show working”, watch the plan run, then find the note in Obsidian.
3. Type *Delete the note <name>* — it must stop and ask for your signature before anything happens.

5 Put it on GitHub

Your portfolio copy. CI runs the tests and evals on every push.

5.1 One-time git setup

```
git config --global user.name "<Your Name>"
git config --global user.email "<the email on your GitHub account>"
```

5.2 Check nothing secret will be committed

`.gitignore` already excludes `.env`, `data/`, `SynapseVault/`, `.venv/`, `web/node_modules/` and the built dashboard. Confirm before the first commit:

```
cd ~/Desktop/synapse
git init -b main
git add .
git status
```

In the list, `.env`, `data/` and `google_credentials.json` must **not** appear. If one does, stop and add it to `.gitignore`, then `git rm --cached <file>`.

```
git commit -m "Synapse: agentic assistant (LangGraph, MCP, CrewAI)"
```

5.3 Create the repository and push

OPTION A — GITHUB CLI (EASIEST)

```
gh auth login # choose GitHub.com, HTTPS, log in with browser
gh repo create synapse --public --source=. --push
```

OPTION B — WEBSITE

1. github.com → **New repository** → name `synapse`. Do *not* tick “Add a README”, `.gitignore` or licence (they would conflict with your first push).
2. Copy the HTTPS URL, then:

```
git remote add origin https://github.com/<your-username>/synapse.git
git push -u origin main
```

When asked for a password, use a **personal access token** (GitHub → Settings → Developer settings → Personal access tokens → Fine-grained → repository access: synapse, Contents: read and write), not your account password.

5.4 CI

`.github/workflows/ci.yml` runs automatically on every push to `main` and on pull requests: lint, the test suite, the evaluation suite (report uploaded as an artifact), the dashboard build and a Docker build. Watch it under the repository's **Actions** tab. No secrets are needed — tests script the model.

Add the status badge to the top of `README.md` :

```
![CI](https://github.com/<your-username>/synapse/actions/workflows/ci.yml/badge.svg)
```

5.5 Everyday workflow

```
git checkout -b feature/<short-name>
# ...edit, then run section 4...
git add -A
git commit -m "<what changed and why>"
git push -u origin feature/<short-name>
gh pr create --fill                # or open the PR on github.com; merge when CI is green
```

5.6 Making it portfolio-ready

- ☐ Repository description and topics: `langgraph`, `mcp`, `crewai`, `agents`, `llmops`, `obsidian`
- ☐ A screenshot or short GIF of a run at the top of the README (Journal with “Show working” open, and the Agents view)
- ☐ A `LICENSE` file (MIT is usual): GitHub → Add file → Create new file → name it LICENSE → choose a template
- ☐ Pin the repository on your profile
- ☐ Search the history once for leaked keys: `git log -p | grep -iE "gsk_|ntn_|api_key|secret"`

IF YOU EVER PUSH A SECRET

Revoke it at the provider immediately (Groq, Notion, Google Cloud), then generate a new one. The old value stays in git history even after you delete the file, and public repositories are scanned by bots within minutes.

6 Fast model (Groq)

Free tier, OpenAI-compatible, 10–30× faster than a local 7B model.

1. Go to **console.groq.com** → sign in → **API Keys** → *Create API Key*. Copy it (starts with `gsk_`).

2. In `.env` :

```
SYNAPSE_FAST_BASE_URL=https://api.groq.com/openai/v1
SYNAPSE_FAST_MODEL=llama-3.3-70b-versatile
SYNAPSE_FAST_API_KEY=<gsk_...>
```

3. Restart the API. The sidebar footer changes from *local* to *fast + local*.

Free tiers have per-minute and per-day limits. When Groq refuses a call, Synapse falls back to Ollama for that call, so a heavy day gets slower rather than broken. Cerebras (<https://api.cerebras.ai/v1> , model `llama-3.3-70b`) works the same way if you want a second option.

PRIVACY TRADE-OFF

With a fast provider, your prompts — including note excerpts used for the task — go to that provider. Leave the three lines unset for fully local operation.

7 Gmail & Google Calendar

About 15 minutes, once. Sending and event creation always ask for your approval.

1. **console.cloud.google.com** → project selector → *New project* → name it `synapse` → Create.

2. **APIs & Services** → **Library**: enable *Gmail API*, then *Google Calendar API*.

3. **OAuth consent screen** (Google Auth Platform → Branding/Audience): User type *External*; app name `Synapse` ; your email as support and developer contact. Under *Audience* → *Test users* add your own Gmail address. Leave the app in *Testing*.

4. **Credentials** → **Create credentials** → **OAuth client ID** → Application type *Desktop app* → Create → *Download JSON*.

5. Move it into the project:

```
mv ~/Downloads/client_secret_*.json ~/Desktop/synapse/data/google_credentials.json
```

6. Authorise (a browser tab opens; choose your account; on “Google hasn't verified this app” click *Continue* — it is your own app):

```
uv run python -m synapse.integrations.google_auth
```

7. Restart the API. `uv run synapse doctor` should print *Google: connected* and the Tools tab lists `gmail_send` , `gmail_search` , `calendar_create_event` ...

TESTING-MODE TOKENS EXPIRE AFTER 7 DAYS

If Gmail tools start failing with an auth error, re-run step 6. To stop the weekly expiry you would need to publish the app, which requires Google verification for Gmail scopes — not worth it for personal use.

Emails are sent as plain text plus clean HTML (tables and lists render properly), never with attachments; the content is written into the body before you approve. Bodies containing placeholder text are refused and the run re-plans to fetch real data.

8 Notion

Five minutes. Synapse can only see pages you explicitly share with it.

1. Open **notion.so/profile/integrations** (older link: notion.so/my-integrations) → *New integration*.
2. Name **Synapse**, pick your workspace, type *Internal* → *Save*.
3. Capabilities: Read content, Update content, Insert content. Copy the **Internal Integration Secret** (starts with `ntn_`).
4. `.env`: `SYNAPSE_NOTION_TOKEN=<ntn_...>` and restart.
5. In Notion, open each page or database Synapse should use → **⋮ (top right)** → **Connections** → **Synapse**. Sharing a parent page shares everything under it.

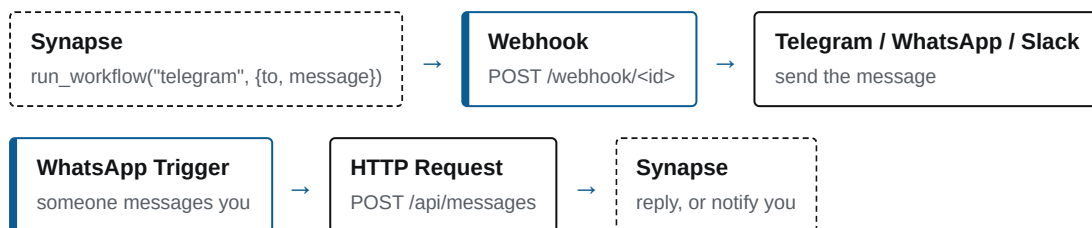
Try: “Add a row to my Reading List database: Designing Data-Intensive Applications, status To read”. Anything not shared returns “not found”, and Synapse says so rather than guessing.

9 n8n: WhatsApp, Telegram, Slack, Sheets

n8n is the bridge to every service Synapse has no built-in tool for.

9.1 How it fits together

An n8n **workflow** is a graph of **nodes** on a canvas, joined left to right. The first node is a *trigger*; each following node receives the previous node's JSON output. Synapse talks to n8n in two directions:



Outbound: Synapse calls `run_workflow(name, payload)`. The name must be registered in `SYNAPSE_N8N_WEBHOOKS`; the payload is always `{"to": "...", "message": "..."} (plus "subject" when relevant)`. **Inbound:** n8n forwards a received message to Synapse's `/api/messages` endpoint.

THREE N8N FACTS THAT TRIP EVERYONE UP

1. **Test URL vs production URL.** Every Webhook node has two. The *test* URL (`/webhook-test/...`) works only while you have clicked *Listen for test event*. The *production* URL (`/webhook/...`) works only when the workflow is **Active** (toggle at the top; newer versions call it *Publish*). Register the production URL.
2. **The POST body is under `body`.** In later nodes the message is `{{ $json.body.message }}`, not `{{ $json.message }}`.
3. **n8n must be able to reach Synapse** for inbound flows (and vice versa for outbound). n8n Cloud cannot reach `localhost` on your Mac — use a tunnel (section 10.4).

9.2 Get n8n

OPTION	HOW	GOOD FOR
n8n Cloud	Sign up at n8n.io (trial, then paid). Public HTTPS out of the box.	WhatsApp/Telegram triggers with no tunnel
On your Mac (npx)	<code>npx n8n</code> → open <code>http://localhost:5678</code>	Trying it; outbound only unless tunnelled
On your Mac (Docker)	<code>docker run -it --rm -p 5678:5678 -v n8n_data:/home/node/.n8n docker.n8n.io/n8nio/n8n</code>	Same, persistent
Your VPS	n8n's Docker Compose + Caddy recipe (docs.n8n.io → Hosting)	Always-on, public triggers

For triggers from WhatsApp or Telegram, n8n needs a public HTTPS URL. Self-hosted on your Mac, start n8n with `WEBHOOK_URL=https://<your-tunnel-host>/` so the URLs it shows are the public ones.

9.3 Secure every Synapse webhook

1. Generate a secret: `openssl rand -hex 24`. Put it in `.env` as `SYNAPSE_N8N_SECRET`.
2. In n8n: **Credentials** → **New** → **Header Auth**. Name: `X-Synapse-Secret`. Value: the secret. Save as “Synapse secret”.
3. In every Webhook node below: **Authentication** → **Header Auth** → **Synapse secret**.

Now a request without the header gets 403 from n8n, even if someone learns the URL.

9.4 Recipe: Telegram messages (start here — free and easy)



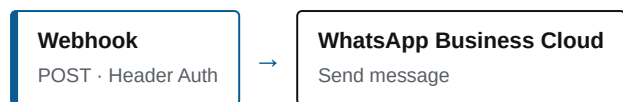
1. In Telegram, message `@BotFather` → `/newbot` → name it → copy the bot token.
2. Send your new bot any message. Then find your chat id: open `https://api.telegram.org/bot<token>/getUpdates` in a browser and copy `message.chat.id`.
3. n8n → **New workflow** → name it “Synapse → Telegram”.
4. Add node **Webhook**: HTTP Method `POST`; Path `synapse-telegram`; Authentication `Header Auth`; Respond When *Last Node Finishes* (so Synapse sees failures).
5. Click + after it → **Telegram** → **Send a text message**. Credential: paste the bot token. Chat ID: your chat id (or `{{ $json.body.to }}` to let Synapse choose). Text: `{{ $json.body.message }}`.
6. Save → **Activate**. Copy the Webhook node's *Production URL*.
7. `.env`: `SYNAPSE_N8N_WEBHOOKS=telegram=<production URL>` → restart Synapse.

Test it without Synapse first (use the secret you generated):

```
curl -X POST "<production URL>" -H "Content-Type: application/json" \
  -H "X-Synapse-Secret: <secret>" -d '{"to":"me","message":"hello from n8n"}'
```

Then in Synapse: “Send me a Telegram message with my open tasks”. It plans `list_tasks` → write → `run_workflow`, and asks for your signature before sending.

9.5 Recipe: WhatsApp (outbound)



1. **developers.facebook.com** → My Apps → *Create app* → use case “Connect with customers through WhatsApp” (type *Business*) → add the **WhatsApp** product.
2. WhatsApp → **API Setup**: note the *Phone number ID* of the free test number, and add your own phone under *To* as an allowed recipient (verify the code sent to you).
3. Create a long-lived token: Business Settings → **System users** → add one → *Generate token* with `whatsapp_business_messaging` and `whatsapp_business_management`.
4. n8n: Webhook node exactly as in 9.4 (path `synapse-whatsapp`) → **WhatsApp Business Cloud** → **Send message**. Credential: access token + business account ID. Sender phone number ID: from step 2. Recipient: `{{ $json.body.to }}` (international format, digits only, e.g. `91XXXXXXXXXX`). Message type *Text*, body `{{ $json.body.message }}`.
5. Activate, copy the production URL, add `whatsapp=<url>` to `SYNAPSE_N8N_WEBHOOKS`.

WHATSAPP'S 24-HOUR RULE

Free-form text can only be sent to someone who messaged your WhatsApp number in the last 24 hours. Starting a conversation cold requires a pre-approved *template* message. For messages to yourself, message the test number once a day, or use Telegram.

9.6 Recipe: Slack



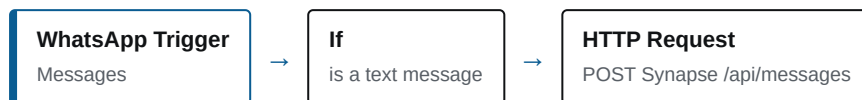
1. api.slack.com/apps → Create New App → From scratch → pick your workspace.
2. OAuth & Permissions → Bot Token Scopes: `chat:write` → *Install to workspace* → copy the `xoxb-` token.
3. In Slack, invite the bot to the channel: `/invite @<app name>`.
4. n8n: Webhook → **Slack** → **Send a message**. Credential: Access Token = the xoxb token. Send message to: Channel `#<channel>` (or `{{ $json.body.to }}`). Text: `{{ $json.body.message }}`.
5. Activate → register as `slack=<production URL>`.

9.7 Recipe: log to Google Sheets



1. Create a sheet with header row `Date | Entry`.
2. n8n: Webhook → **Edit Fields (Set)**: `Date = {{ $now.toISOString() }}`, `Entry = {{ $json.body.message }}` → **Google Sheets** → **Append row** (sign in with Google; pick the document and sheet; map columns automatically).
3. Register as `sheet=<production URL>`. Try: *“Log to my sheet: finished LangGraph chapter 3”*.

9.8 Recipe: inbound WhatsApp auto-reply



1. Synapse must be reachable from n8n: same machine (`http://localhost:8000` ; from n8n in Docker use `http://host.docker.internal:8000`), or a tunnel URL (section 10.4). Set `SYNAPSE_API_TOKEN`.
2. n8n → new workflow → **WhatsApp Trigger**, Updates: `messages`, credential = the Meta app (client ID/secret from the app's Basic settings). n8n registers the callback with Meta when you activate.
3. **If** node: condition `{{ $json.messages[0].type }}` is equal to `text` (ignores read receipts and media). Open the trigger's output panel once to confirm these field names — Meta sometimes nests them differently.
4. **HTTP Request** (true branch): Method `POST` ; URL `<synapse>/api/messages` ; Authentication: Generic → *Header Auth* with name `Authorization` , value `Bearer <SYNAPSE_API_TOKEN>` ; Send Body → JSON:

```
{
  "source": "whatsapp",
  "sender": "{{ $json.messages[0].from }}",
  "text":  "{{ $json.messages[0].text.body }}"
}
```

5. Activate. The outbound `whatsapp` workflow from 9.5 must also be registered — Synapse replies through it.

What happens next: Synapse classifies the message. Routine (greeting, thanks, simple question, confirming a time): it drafts a short reply in your voice and sends it with `run_workflow("whatsapp", {to: sender, message})`. Anything

involving money, commitments, sensitive topics or an upset tone: *no reply* — you get a notification with a suggested draft. The incoming text is treated strictly as data, never as instructions.

REPLIES WAIT FOR YOUR ONE-CLICK APPROVAL BY DEFAULT

`run_workflow` is classed high-risk, so each reply appears under “⚠ to sign” in the dashboard. To let routine replies go out on their own, change `"run_workflow": "high"` to `"medium"` in `synapse/guardrails.py` (medium still asks whenever the content came from the web). Do this only once you trust its judgement on a week of real messages.

Telegram works the same way: **Telegram Trigger** → HTTP Request with `"source": "telegram"`, `"sender": "{{ $json.message.chat.id }}"`, `"text": "{{ $json.message.text }}"`, and a registered `telegram` workflow for replies.

9.9 n8n troubleshooting

SYMPTOM	FIX
“workflow 'x' is not registered”	Name missing from <code>SYNAPSE_N8N_WEBHOOKS</code> , or you did not restart Synapse after editing <code>.env</code> .
HTTP 404 from n8n	Workflow not Active, or you registered the <code>/webhook-test/</code> URL.
HTTP 403 from n8n	Secret mismatch between <code>SYNAPSE_N8N_SECRET</code> and the Header Auth credential (name must be exactly <code>X-Synapse-Secret</code>).
Message arrives empty	Expression used <code>\$json.message</code> ; it must be <code>\$json.body.message</code> .
Synapse says sent, nothing arrived	Webhook set to “Respond Immediately”, so errors were hidden. Use “When Last Node Finishes” and check n8n’s <i>Executions</i> list.
Inbound never reaches Synapse	n8n cannot reach the URL (localhost from the cloud), or 401 from a missing/incorrect Bearer token.

10 Deployment

Pick one. The first is the recommended default for a personal assistant.

ROUTE	COST	MODEL	REACHABLE FROM PHONE	CHOOSE WHEN
10.1 Mac, always on	Free	Local + Groq	With a tunnel	Default. Private, your vault stays local.
10.2 Docker (Mac)	Free	Local (host) + Groq	With a tunnel	You want a container you can rebuild.
10.3 VPS + Caddy	~\$5/month	Groq only	Yes	Must work while your Mac sleeps.

10.1 Your Mac, always on

A launch agent starts Synapse at login and restarts it if it crashes. Replace `<you>` twice:

```
cat > ~/Library/LaunchAgents/com.synapse.api.plist <<'PLIST'
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE plist PUBLIC "-//Apple//DTD PLIST 1.0//EN" "http://www.apple.com/DTDs/PropertyList-1.0.dtd">
<plist version="1.0"><dict>
  <key>Label</key><string>com.synapse.api</string>
  <key>ProgramArguments</key>
  <array><string>/Users/<you>/Desktop/synapse/.env/bin/synapse-api</string></array>
  <key>WorkingDirectory</key><string>/Users/<you>/Desktop/synapse</string>
  <key>RunAtLoad</key><true/>
  <key>KeepAlive</key><true/>
  <key>StandardOutPath</key><string>/tmp/synapse.log</string>
  <key>StandardErrorPath</key><string>/tmp/synapse.err</string>
</dict></plist>
PLIST

launchctl load ~/Library/LaunchAgents/com.synapse.api.plist
brew services start ollama
```

- Logs: `tail -f /tmp/synapse.err`. Stop: `launchctl unload ~/Library/LaunchAgents/com.synapse.api.plist`.
- After changing `.env` or pulling code: unload, then load again.
- Scheduled goals run only while the Mac is awake: System Settings → Battery → Options → *Prevent automatic sleeping when the display is off* (on power adapter).

10.2 Docker

```
cp .env.example .env # fill it in (section 3)
VAULT_PATH="/Users/<you>/Documents/<YourVault>" docker compose up -d --build
docker compose logs -f
curl http://127.0.0.1:8000/api/health
```

- `docker-compose.yml` reads your `.env`, then fixes the paths inside the container (`/vault`, `/data`) and points at Ollama on the host.

- The vault is a bind mount: notes land on your disk and Obsidian sees them live. `./data` holds all state.
- The port is published on `127.0.0.1` only. Add the crew with `SYNAPSE_EXTRAS=google,crew` `docker compose up -d --build`.
- Update: `git pull && docker compose up -d --build`. The healthcheck marks the container unhealthy if the API stops answering.

10.3 VPS with HTTPS

For a small Ubuntu VM (1 vCPU / 2 GB is enough; any provider). You need a domain or subdomain pointing at its IP (DNS A record, e.g. `synapse.<yourdomain>`).

```
# on the server, as a sudo user
sudo apt update && sudo apt install -y docker.io docker-compose-v2 git caddy
sudo usermod -aG docker $USER && newgrp docker
git clone https://github.com/<your-username>/synapse.git && cd synapse
cp .env.example .env && nano .env          # VPS values from 3.8; chmod 600 .env
mkdir -p vault && VAULT_PATH=./vault docker compose up -d --build
```

Caddy obtains and renews the TLS certificate automatically. `/etc/caddy/Caddyfile`:

```
synapse.<yourdomain> {
    reverse_proxy 127.0.0.1:8000
}
```

```
sudo systemctl reload caddy
sudo ufw allow OpenSSH && sudo ufw allow 80,443/tcp && sudo ufw enable
```

THE VAULT ON A SERVER

On a VPS the vault is a folder on the server, not the one Obsidian opens on your Mac. Sync it (Obsidian Sync, Syncthing, or a private git repo) if you want both. This is why 10.1 is the default.

10.4 Tunnels: reach your Mac from your phone or from n8n

QUICK, TEMPORARY URL

```
brew install cloudflared
cloudflared tunnel --url http://localhost:8000    # prints https://<random>.trycloudflare.com
```

The URL changes each time — fine for testing, not for n8n.

STABLE URL (FREE, NEEDS A DOMAIN ON CLOUDFLARE)

```
cloudflared tunnel login
cloudflared tunnel create synapse
cloudflared tunnel route dns synapse synapse.<yourdomain>
cloudflared tunnel run --url http://localhost:8000 synapse
sudo cloudflared service install          # keep it running across reboots
```

ngrok works too: `ngrok http 8000` (a free static domain is available in its dashboard).

ONE TOKEN, ONE USER

Synapse is single-user: `SYNAPSE_API_TOKEN` is the whole authorisation model, with no accounts or roles. Anyone with the URL and the token can read your vault and send mail as you. Share neither.

11 Pre-launch checklist

Tick every line before the port is reachable by anything but your Mac.

CORRECTNESS

- ☐ `uv run pytest -q` passes
- ☐ `uv run python -m evals.run` passes
- ☐ `uv run ruff check synapse evals tests` clean
- ☐ `npm run build` without warnings
- ☐ `uv run synapse doctor` shows the right vault
- ☐ CI green on GitHub

OPERATIONS

- ☐ `data/` and the vault are backed up
- ☐ `/api/health` returns 200
- ☐ You know how to roll back (12.3)
- ☐ You know where the logs are

SECURITY

- ☐ `SYNAPSE_API_TOKEN` is a long random value
- ☐ `SYNAPSE_HOST=127.0.0.1`; access only via tunnel or Caddy
- ☐ HTTPS on every public URL
- ☐ `.env`, `data/` never committed; `chmod 600 .env` on servers
- ☐ Approvals left on for email, deletes and workflows
- ☐ `SYNAPSE_N8N_WEBHOOKS` lists only workflows you built
- ☐ `SYNAPSE_N8N_SECRET` set and checked in n8n

FIRST HOUR AFTER GOING LIVE

- `curl https://<your-url>/api/health` from your phone's network returns 200.
- `curl https://<your-url>/api/runs` without the token returns 401.
- Run one real goal and approve one risky action end to end.
- Check the trace (Show working) recorded it and the Evals tab still loads.

12 Operating it

Backups, upgrades, rollback, and what normal looks like.

12.1 Back up

PATH	CONTENTS	HOW
your vault	Every note Synapse wrote or edited	Time Machine / Obsidian Sync
<code>data/</code>	Runs, traces, memory, schedules, checkpoints, Google token	Time Machine, or <code>tar czf</code> nightly
<code>.env</code>	Keys and paths	Password manager

Deleted notes go to `.trash/` inside the vault, so an accidental delete is recoverable.

12.2 Upgrade

```
git pull
uv sync --extra dev --extra google
cd web && npm install && npm run build && cd ..
uv run pytest -q && uv run python -m evals.run
launchctl unload ~/Library/LaunchAgents/com.synapse.api.plist && launchctl load
~/Library/LaunchAgents/com.synapse.api.plist
```

12.3 Roll back

SITUATION	ACTION
Bad release	<code>git log --oneline</code> → <code>git checkout <good-commit></code> → rebuild → restart
A feature misbehaves	<code>SYNAPSE_ENABLE_WEB/CREW/SCHEDULER=false</code> , remove an n8n webhook, restart
Corrupt state	Stop, restore <code>data/</code> from backup, start
Provider quota or cost	Remove the <code>SYNAPSE_FAST_*</code> lines; it runs fully local

12.4 What normal looks like

ACTION	LOCAL 7B	WITH GROQ
Greeting / general question	2–6 s	< 2 s
Note written and verified	30–70 s	5–15 s
Research with sources	1–3 min	20–50 s
Crew research report	3–6 min	1–2 min

Every run records node latencies, model, tokens, tool calls, approvals and errors — visible under “Show working” and stored in `data/traces.db`.

13 Troubleshooting

Symptom, then cause and fix.

SYMPTOM	CAUSE AND FIX
command not found: uv	brew install uv , then open a new terminal.
zsh: no matches found: .[dev]	Quote extras in zsh: "[dev]". With uv sync --extra dev this cannot happen.
doctor: Ollama unreachable	ollama serve is not running (or brew services start ollama).
0 notes , nothing in Obsidian	SYNAPSE_VAULT_PATH is not the vault Obsidian opens; compare with doctor's output.
Page shows “Can't reach the Synapse API”	API not running, or a different port. Check /tmp/synapse.err .
Blank dashboard	Dashboard not built after a change: cd web && npm run build . The browser console names any error.
401 everywhere after setting a token	The dashboard asks for the token once; clear site data if an old one is stored.
Runs take minutes	Local 7B. Add Groq (section 6).
Groq errors / rate limit	Free-tier limit; it falls back to Ollama. Check the key and model name.
Gmail tools fail after a week	Testing-mode token expired. Re-run uv run python -m synapse.integrations.google_auth .
Email bounced (“address not found”)	Check the To line in the approval card before signing. Labels glued to Gmail addresses (mail-you@gmail.com) are removed automatically.
“recipient has not been used before”	Include the address once in your message; after that it is remembered.
Email refused: placeholder text	Working as intended — the data wasn't fetched. It re-plans; if it still fails, name the city for the weather.
Notion: not found	The page isn't shared with the integration (⋯ → Connections).
Scheduled goal never ran	API wasn't running or the Mac was asleep at that time.
A server “unavailable” in the sidebar	That MCP server failed to start; uv run synapse doctor prints why.
n8n issues	See 9.9.

14 Changing it yourself

Where to edit for the changes you are most likely to want.

CHANGE	WHERE
Add a tool	A function in synapse/mcp_servers/*.py (raise ToolError with an actionable message) → its risk in RISK_POLICY (synapse/guardrails.py ; unlisted tools default to high) → READ_ONLY if safe to retry → UNTRUSTED_TOOLS if it returns outside content → an eval in evals/scenarios.py .
What needs approval	RISK_POLICY : high always asks, medium asks when input is untrusted, low never asks.
Planner / writer behaviour	PLANNER_RULES and WRITER_RULES in synapse/graph.py . Hard rules belong in validate_plan() , which forces the planner to correct itself.

Agents shown in the UI	<code>ROSTER</code> in <code>web/src/panels.jsx</code> — give an agent a <code>tools</code> list and it lights up when they run.
Colours and look	<code>web/src/index.css</code> : black, greys and one sky-green accent (<code>--signal</code>) for live state, approvals and agents. Light mode: off-white page with light-grey and silver surfaces; dark mode: grey page; the sidebar is always black (<code>.surface-dark</code>). One typeface (IBM Plex Sans) — hierarchy from weight, italics and case. Rebuild the dashboard after changes.
Email formatting	<code>to_html()</code> / <code>to_plain_text()</code> in <code>synapse/mcp_servers/google.py</code> .
Checks	Behaviour: <code>evals/scenarios.py</code> . UI: <code>scripts/browser_check.py</code> . Unit: <code>tests/</code> .
This guide	<code>scripts/deploy_doc.html</code> → <code>uv run python scripts/make_deploy_pdf.py</code> .

HONEST LIMITS

Single user, one token. Guardrails reduce risk but are not a security boundary — MCP servers run with your permissions. Verification proves a write landed exactly; it does not prove a summary is true. Gmail, Calendar, Notion and the n8n recipes follow each service's documented setup but were not exercised against your real accounts here — treat the first run of each as a test. Full list: `docs/LIMITATIONS.md` .